

CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES:

Introduction:

The StyleCart Fashion E-Commerce Order Management Database System is designed to store and manage customer, product, cart, order, payment, and review information efficiently. Analyzing functional dependencies, data redundancy, and data anomalies helps ensure that the database is well-structured, consistent, and free from unnecessary duplicate data. Functional dependency defines the relationship between attributes, redundancy analysis identifies and reduces duplicate data, and data anomaly analysis checks for insertion, update, and deletion problems. Since the database is properly normalized, it minimizes redundancy and prevents anomalies, resulting in improved data integrity, consistency, and overall database performance.

1. CUSTOMER TABLE:

Functional Dependency

Customer_ID → First_Name, Last_Name, Email, Phone, Password, Address, City, State, Pincode

Redundancy Analysis

Customer information is stored only once using Customer_ID as the primary key, thereby minimizing data redundancy.

Data Anomalies

No data anomalies were identified.

Reason: Customer information is uniquely identified by Customer_ID and all customer details are stored in a single table. The normalized design prevents insert, update, and delete anomalies.

2. BRAND TABLE:

Functional Dependency

Brand_ID → Brand_Name, Description, Country

Redundancy Analysis

Brand information is stored only once using Brand_ID as the primary key, thereby minimizing data redundancy.

Data Anomalies

No data anomalies were identified.

Reason: Brand information is uniquely identified by Brand_ID and stored only once. The normalized design prevents insert, update, and delete anomalies.

3. CATEGORY TABLE:

Functional Dependency

Category_ID → Category_Name, Description

Redundancy Analysis

Category information is stored only once using Category_ID as the primary key, thereby minimizing data redundancy.

Data Anomalies

No data anomalies were identified.

Reason: Category information is uniquely identified by Category_ID and maintained in a separate table. The normalized design prevents insert, update, and delete anomalies.

4. PRODUCT TABLE:

Functional Dependency

Product_ID → Product_Name, Brand_ID, Category_ID, Price, Size, Color, Material, Stock, Product_Image

Redundancy Analysis

Product information is stored only once using Product_ID as the primary key. Brand and Category information are maintained in separate tables and referenced using foreign keys, thereby minimizing data redundancy.

Data Anomalies

No data anomalies were identified.

Reason: Product information is uniquely identified by Product_ID and linked to the Brand and Category tables using foreign keys. The normalized design prevents insert, update, and delete anomalies.

5. CART TABLE:

Functional Dependency

Cart_ID → Customer_ID, Grand_Total, Total_Items

Redundancy Analysis

Cart information is stored only once using Cart_ID as the primary key and linked to the Customer table through a foreign key, thereby minimizing data redundancy.

Data Anomalies

No data anomalies were identified.

Reason: Cart information is uniquely identified by Cart_ID and linked to the Customer table using a foreign key. The normalized design prevents insert, update, and delete anomalies.

6. CART_ITEM TABLE:

Functional Dependency

CartItem_ID → Cart_ID, Product_ID, Quantity, Total_Price

Redundancy Analysis

Cart item information is stored only once using CartItem_ID as the primary key and references the Cart and Product tables through foreign keys, thereby reducing duplicate entries.

Data Anomalies

No data anomalies were identified.

Reason: Cart item information is uniquely identified by CartItem_ID and linked to the Cart and Product tables using foreign keys. The normalized design prevents insert, update, and delete anomalies.

7. ORDER TABLE:

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status, Shipping_Address

Redundancy Analysis

Order information is stored only once using Order_ID as the primary key and linked to the Customer table through a foreign key, thereby minimizing data redundancy.

Data Anomalies

No data anomalies were identified.

Reason: Order information is uniquely identified by Order_ID and linked to the Customer table using a foreign key. The normalized design prevents insert, update, and delete anomalies.

8. PAYMENT TABLE:

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Date, Payment_Status, Amount_Paid

Redundancy Analysis

Payment information is stored only once using Payment_ID as the primary key and linked to the Order table through a foreign key, thereby minimizing data redundancy.

Data Anomalies

No data anomalies were identified.

Reason: Payment information is uniquely identified by Payment_ID and linked to the Order table using a foreign key. The normalized design prevents insert, update, and delete anomalies.

9. REVIEW TABLE:

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Comment, Review_Date

Redundancy Analysis

Review information is stored only once using Review_ID as the primary key and linked to the Customer and Product tables through foreign keys, thereby minimizing data redundancy.

Data Anomalies

No data anomalies were identified.

Reason: Review information is uniquely identified by Review_ID and linked to the Customer and Product tables using foreign keys. The normalized design prevents insert, update, and delete anomalies.

CONCLUSION:

The StyleCart Fashion E-Commerce Database Management System is designed using proper functional dependencies and normalized tables. Each entity stores only its relevant information, reducing redundancy and preventing insert, update, and delete anomalies. The database maintains data consistency, improves integrity, and supports efficient management of

customers, products, shopping carts, orders, payments, and reviews. This design provides a reliable and scalable database structure for an online fashion e-commerce platform.